FINAL-RUN · 5

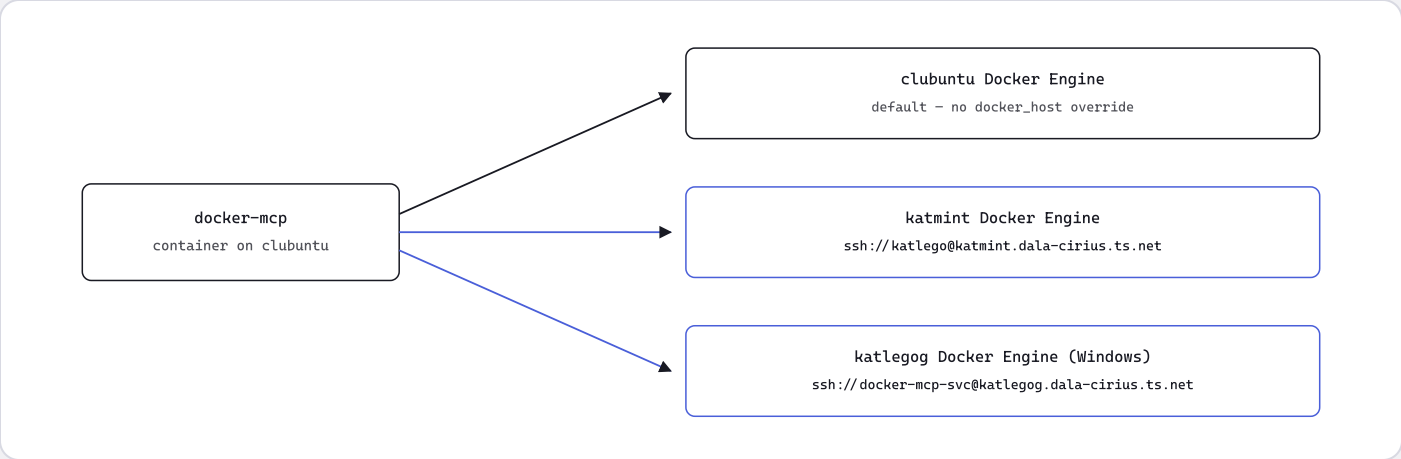
docker-mcp remote engines

The harder half of the pair promised in §07 of the previous doc: docker-mcp on clubuntu can now reach into Docker engines on other tailnet machines, per call, over SSH — the second and last of the two MCP servers meant to be pointable at any host.

tool: list-containers, docker_host override 2026-08-16

00 The shape of it

One `docker-mcp` server, running on clubuntu, now answers for three different Docker engines depending on one argument.



Same tool, same code path, three destinations — the argument is the only thing that changes.

01 What was already true

Same shape as the postgres-mcp doc: checking the source before changing anything found the mechanism already built. `create-container`, `deploy-compose`, `get-logs`, and `list-containers` all already accept an optional `docker_host` argument. When given, `get_docker_client()` in `handlers.py` builds a throwaway `python-on-whales` `DockerClient` scoped to that one call instead of reusing the shared default client — deliberately restricted to the `ssh://` scheme only, so no TLS certificate or key material

ever has to pass through a tool argument. Zero application code changes needed here either.

The actual gap, same pattern as before: infrastructure. No SSH client in the image, no key or `known_hosts` to use one with, and no target host credentialed yet.

02 Credentialing two targets

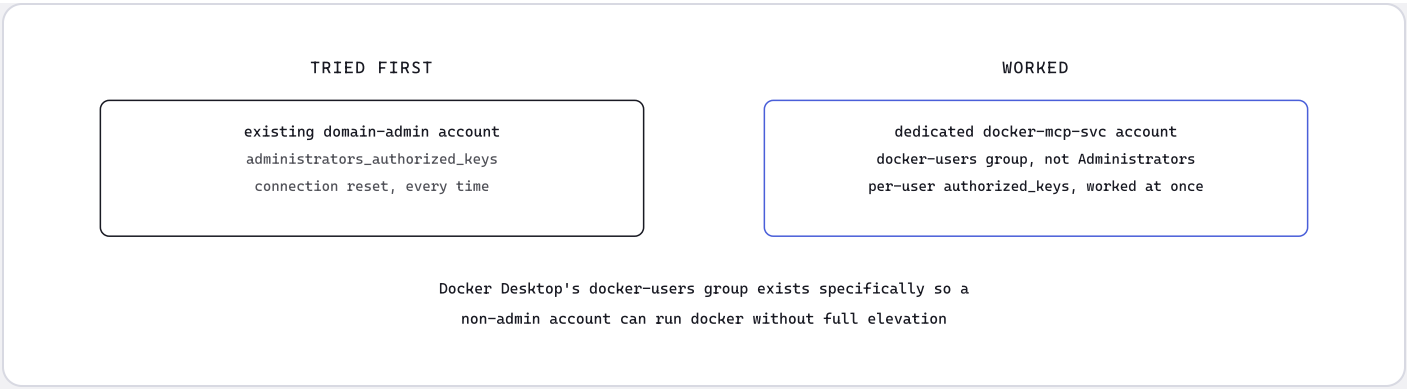
One dedicated ed25519 keypair, generated on clubuntu itself (private half never leaves that machine, never committed), reused across every target host rather than minting one per host.

TARGET	RESULT
`katmint`	**✓ appended to existing `katlego` user's `authorized_keys` — already in the `docker` group, worked immediately**
`katlegog` (this laptop, Windows)	*needed a detour — see §03*

03 An honest detour: Windows admin accounts and SSH

The obvious move on katlegog — add the new public key to the existing (admin) Windows account — failed silently. Key exchange completed fine; the connection reset the instant the client sent its first auth request, every time, with no further detail even after cranking `sshd`'s `LogLevel` to `DEBUG3`.

Root cause: on Win32-OpenSSH, a connecting account that belongs to the local `Administrators` group is routed through `Match Group administrators` to a system-level `administrators_authorized_keys` file with strict ACL requirements — a well-known source of exactly this kind of silent reset, and this laptop's account is a *domain* admin, which compounds it further.



Profile directory for the new account doesn't exist until first logon — forced with `Start-Process -Credential $cred -LoadUserProfile` rather than a real interactive session.

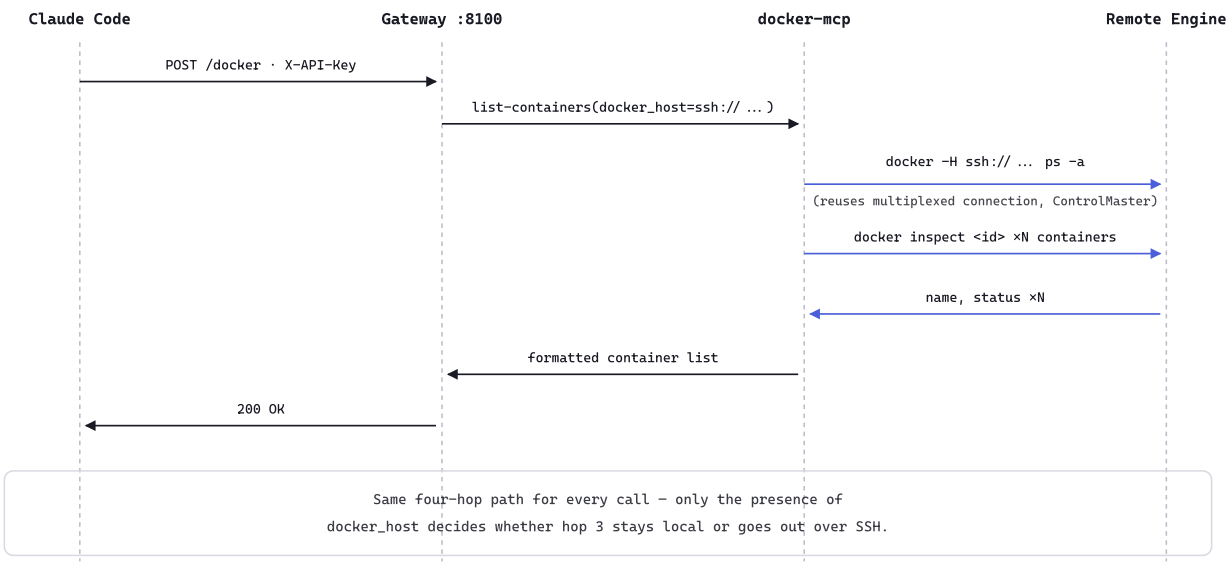
04 Implementation

FILE	CHANGE
`docker-mcp/Dockerfile`	installs `openssh-client`; bakes in `ssh_config`; creates `/root/.ssh/control` for connection multiplexing (§07)
`docker-mcp/ssh_config`	`IdentityFile`, `UserKnownHostsFile`, `StrictHostKeyChecking yes` — trust established once on the host running docker-mcp, never accepted from inside the container
`docker-compose.yml` / `docker-compose.deploy.yml`	`docker-mcp` service mounts the private key and clubuntu's own `known_hosts` read-only, alongside the existing Docker socket mount
`.env.example`	documents `DOCKER_MCP_SSH_KEY_PATH` / `DOCKER_MCP_SSH_KNOWN_HOSTS_PATH` and the setup steps for adding a future third host
`build.yml`	throwaway `/dev/null` CI values for the two new vars — same reason `XLSX_HOST_PATH` needed one: they're bind-mount sources, and `docker compose build` parses the whole file even without running it

Deployed as `v0.1.3` via the existing tag-triggered pipeline: push to `main` builds the image, a `v*` tag redeploys it on clubuntu.

05 Proof: three engines, one tool

Confirmed directly on the running container before touching the MCP layer — its mounts now show the key and known_hosts, not just the socket — then exercised the real way, through `list-containers`, no different from any other tool call except one argument. The full path for the remote case has three hops, not one:



One call, four hops — the only thing that changes between targets is whether hop three is a local socket or an SSH-tunneled remote engine.

```
LIST-CONTAINERS(DOCKER_HOST=...)
```

```
list-containers()
  → clubuntu's own containers (default engine, unchanged)

list-containers(docker_host="ssh://katlego@katmint.dala-cirius.ts.net")
  → postgres_db, influxdb3-core running; 31 others exited

list-containers(docker_host="ssh://docker-mcp-svc@katlegog.dala-cirius.ts.net")
  → codercommands-postgres, postgres-mcp, pgadmin, mcp-gateway …  running; 28 others exited
```

Three distinct container lists, from three distinct machines, through the exact same tool — the only thing distinguishing them was one argument, same as postgres-mcp's `connection_uri`.

06 Global aliases

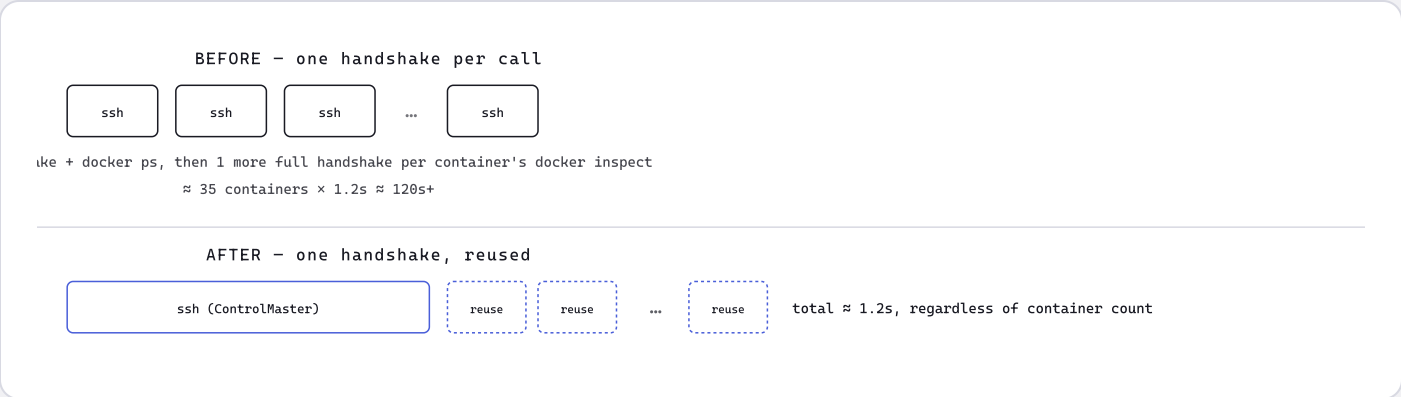
Rather than typing full `ssh://user@host` strings each time, `katmint` and `katlegog` are now recognized shorthand for their respective `docker_host` values — defined in the user-level `~/.claude/CLAUDE.md`, not this repo, so they resolve from any Claude Code session on any project, not just this one.

07 A second honest detour: 120 seconds for a container list

`list-containers` against either remote host was taking well over two minutes — long enough to get moved to a background task both times. A raw, single `ssh … docker ps` round-trip to either host measured about the same, ~1.2 seconds — so the hosts themselves weren't the bottleneck.

The real cause: `python-on-whales`'s `container.list(all=True)` returns lightweight proxy objects, and reading `.name` or `.state.status` on each — which the handler does for every container in the list — lazily triggers a separate `docker inspect` CLI call per container. With no SSH connection reuse configured, each of those paid its own full handshake. ~35 containers per host × ~1.2s per handshake lines up almost exactly with the delay observed.

MEASUREMENT	RESULT
Single `ssh … docker ps`, katmint	`1.173s`
Single `ssh … docker ps`, katlegog	`1.217s`
`list-containers` via MCP, before the fix	*120s+, backgrounded, both hosts*
`list-containers` via MCP, after the fix	**✓ immediate, both hosts**



The handshake is the expensive part, not the command — pay it once per call instead of once per container.

Fix: `ControlMaster auto` / `ControlPath /root/.ssh/control/%r@%h:%p` / `ControlPersist 60s` added to `ssh_config`, with the control-socket directory created in the Dockerfile. All the per-container `docker inspect` calls within one tool call — and back-to-back calls within 60 seconds of each other — now reuse a single already-authenticated connection instead of renegotiating SSH dozens of times. Deployed as

`v0.1.4`; re-running the same katlegog call that had been backgrounded came back instantly.

08 Where this leaves things

- Both MCP servers meant to be pointable at any machine — `postgres-mcp` (§04 of the previous doc) and `docker-mcp` (this one) — are now proven end to end, deployed, and fast.
- Adding a third target host to either is now a known, repeatable recipe: credential it (dedicated non-admin account if it's Windows), pin its host key into the mounted `known_hosts`, and it's reachable — no image rebuild required for `postgres-mcp`, no code changes required for either.
- The Windows-admin-account SSH pitfall from §03 is worth remembering the next time any automated/service SSH access is needed into a Windows host, in this project or elsewhere.
